

JWT in Django Setup

U You 5/24/2026, 3:44:44 AM

i want to learn jwt in django.
teach me.

1. make virtual env
2. make the apps.
3. appA
4. it has a template and static folder.
5. 1st render the login form and then give the access after jwt authentication to dashboard

AI ChatGPT 5/24/2026, 3:44:45 AM

1. Create Project Folder

```
mkdir jwt_django  
cd jwt_django
```

2. Create Virtual Environment

Windows

```
python -m venv venv  
venv\Scripts\activate
```

Linux / Mac

```
python3 -m venv venv  
source venv/bin/activate
```

3. Install Django + JWT Package

```
pip install django djangorestframework djangorestframework-simplejwt
```

4. Create Django Project

```
django-admin startproject config .
```

5. Create App

```
python manage.py startapp appA
```

Final Structure

```
jwt_django/  
  
  appA/  
    templates/  
      login.html  
      dashboard.html  
  
    static/  
      css/  
  
    views.py  
    urls.py  
  
  config/  
    settings.py  
    urls.py  
  
  manage.py
```

6. Add Apps in [settings.py](#).

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',  
    'appA',  
]
```

7. Configure Templates

In settings.py

```
TEMPLATES = [  
    {  
        ...  
        'DIRS': [],  
        ...  
    },  
]
```

Django will automatically detect:

```
appA/templates/
```

8. Configure Static Files

In settings.py

```
STATIC_URL = 'static/'
```

9. JWT Configuration

Add in settings.py

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': (  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ),  
}
```

10. Create Login Template

appA/templates/login.html

```
<form method="POST">
  {% csrf_token %}

  <input type="text" name="username" placeholder="username">

  <input type="password" name="password" placeholder="password">

  <button type="submit">Login</button>
</form>
```

11. Create Dashboard Template

appA/templates/dashboard.html

```
<h1>Dashboard</h1>

<p>You are authenticated using JWT.</p>
```

12. Understand the Flow

JWT login flow:

User fills login form

Django verifies username/password

JWT Access Token generated

Token stored

Protected page checks token

Dashboard access granted

13. Create User

```
python manage.py migrate  
python manage.py createsuperuser
```

14. Basic Views Logic

You need 3 views:

A. Login Page View

Purpose:

- Render login form
 - Accept POST data
 - Authenticate user
-

B. Generate JWT Token

After successful authentication:

```
RefreshToken.for_user(user)
```

This generates:

- access token
 - refresh token
-

C. Dashboard View

Protected route.

It should:

- verify JWT token
 - allow access only if token valid
-

15. URLs

You need:

```
/
  login page

/dashboard/
  protected dashboard
```

16. Main JWT Concept

JWT token looks like:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6I...
```

It contains:

- user id
- expiry
- signature

Server verifies signature before giving access.

17. Important Learning

Traditional Session Auth

Server stores session

JWT Auth

Client stores token

Usually:

- localStorage
 - cookies
 - Authorization header
-

18. Simple JWT Login Logic

Conceptually:

```
user = authenticate(username, password)
```

```
if user:  
    token = generate_jwt_token(user)
```

19. Protected Route Logic

Conceptually:

```
if valid_jwt_token:  
    show_dashboard  
else:  
    deny_access
```

20. What You Learn Here

After this project you will understand:

- Django auth system
- JWT basics
- Access token
- Refresh token
- Protected routes
- Login flow
- Authentication middleware
- Session vs JWT

- API authentication basics

U You 5/24/2026, 4:00:59 AM

done till step13 , i dont understand it. do the handholding and guiding

AI ChatGPT 5/24/2026, 4:01:00 AM

Good.

Now starts the real JWT part.

Goal

You already have:

- project
- appA
- templates
- user

Now we will:

1. Open login page
2. Submit username/password
3. Verify user
4. Generate JWT token

5. Save token
6. Open dashboard only if token valid

STEP 14 — Install Imports

Open:

```
appA/views.py
```

Add:

```
from django.shortcuts import render, redirect  
  
from django.contrib.auth import authenticate  
  
from rest_framework_simplejwt.tokens import RefreshToken
```

Understand Before Writing

This line:

```
authenticate(username,password)
```

checks:

```
Does this user exist in database?  
Is password correct?
```

If correct:

```
returns User object
```

Else:

```
returns None
```

STEP 15 — Create Login View

Inside views.py

```
def login_view(request):
```

```
if request.method == "POST":

    username = request.POST.get("username")

    password = request.POST.get("password")

    user = authenticate(
        username=username,
        password=password
    )

    if user is not None:

        refresh = RefreshToken.for_user(user)

        access_token = str(refresh.access_token)

        request.session['access_token'] = access_token

        return redirect('/dashboard/')

    return render(request, 'login.html')
```

VERY IMPORTANT UNDERSTANDING

This is the entire JWT generation:

```
refresh = RefreshToken.for_user(user)
```

It creates:

```
refresh token  
access token
```

Then:

```
access_token = str(refresh.access_token)
```

extracts access token.

Question:

Where are we storing token?

Here:

```
request.session['access_token']
```

For learning purpose only.

Later you will learn:

- cookies
 - localStorage
 - Authorization header
-

STEP 16 — Create Dashboard View

Below login view:

```
def dashboard(request):  
  
    token = request.session.get('access_token')  
  
    if not token:  
  
        return redirect('/')  
  
    return render(request, 'dashboard.html')
```

Understand This

This line:

```
request.session.get('access_token')
```

means:

Does user have JWT token?

If NO:

Go back to login

If YES:

Allow dashboard

IMPORTANT

Currently we are ONLY checking:

token exists or not

NOT verifying validity yet.

First understand flow.

Then later:

- token validation
 - expiry
 - middleware
 - API protection
-

STEP 17 — Create URLs

Open:

```
appA/urls.py
```

Create file if not exists.

Add:

```
from django.urls import path

from .views import login_view, dashboard

urlpatterns = [

    path("", login_view),

    path('dashboard/', dashboard),
]
```

STEP 18 — Connect App URLs

Open:

```
config/urls.py
```

Add:

```
from django.urls import path, include
```

Then:

```
urlpatterns = [  
    path("", include('appA.urls')),  
]
```

STEP 19 — Run Server

```
python manage.py runserver
```

STEP 20 — Open Browser

Go:

```
http://127.0.0.1:8000/
```

You should see login form.

STEP 21 — Login

Use:

```
superuser username  
superuser password
```

What Happens Internally

Before Login

```
No token
```

After Successful Login

authenticate()

user valid

JWT generated

stored in session

redirect dashboard

VERY IMPORTANT JWT UNDERSTANDING

This token:

eyJhbGc...

is basically:

digital identity card

Server says:

"Yes this user is genuine"

using cryptographic signature.

Current Architecture

Browser

Login Form

Django authenticate()

JWT Generated

Stored

Dashboard Access

NEXT THINGS YOU SHOULD LEARN

After this works, next steps are:

1. Decode token
2. Verify token properly
3. Token expiry
4. Refresh token
5. Protect APIs
6. JWT middleware
7. Authorization header
8. SPA frontend auth

U You 5/24/2026, 4:14:04 AM

also now, i have many records inside the db.
how to do jwt for them? dont code just tell

AI ChatGPT 5/24/2026, 4:14:06 AM

Your database records are unrelated to JWT directly.
JWT is only about:

WHO is accessing the system

not:

WHAT data exists in database

Think Like This

Database Records

Products
Orders
Students
Employees
Messages
Invoices

These are application data.

JWT

JWT only carries:

user identity

Example:

User ID = 5
Username = admin
Role = manager

Real Flow

User logs in

JWT generated

User requests data

Server checks token

If valid:
 allow database access

Example

Suppose database has:

10000 products
500 users
200 invoices

JWT does NOT store all this.

JWT only says:

"This request is from user X"

Then How Data Access Happens?

After token verification:

token valid

server knows user identity

server fetches records from DB

Example:

User ID 7

Fetch invoices of user 7

Important Concept

JWT replaces:

session identity management

NOT database.

In Big Systems

JWT commonly stores:

user_id
email
role
permissions
expiry

Then backend uses those values to:

query database

Example Architecture

Frontend

JWT Token

Backend verifies token

Backend queries DB

Returns records

Role-Based Access

Example:

admin
manager

customer
staff

JWT can carry role.

Then backend decides:

admin all records
customer only own records

Very Important

JWT is NOT:

- database
- storage
- cache
- ORM
- query system

JWT is ONLY:

authentication identity proof

Simple Analogy

JWT is like:

movie ticket

The theater database still exists.

Ticket only proves:

you are allowed inside

U You 5/24/2026, 4:15:11 AM

ok for 500 users ? will i need to make the superuser create ?

AI ChatGPT 5/24/2026, 4:15:12 AM

No.

For 500 users you do NOT create 500 superusers.

Superuser Purpose

Superuser is only for:

admin control

Like:

- Django admin panel
- managing users
- managing database
- full permissions

Usually:

1 or few superusers only

Normal Users

For actual application users:

customers
students

employees
clients

you create NORMAL users.

Flow

User registers

Record stored in auth_user table

User logs in

JWT generated

So For 500 Users

You will have:

1 admin superuser
500 normal users

Django Has Built-in User Model

Each user record stores:

```
username  
password  
email  
is_staff  
is_superuser
```

JWT Works For ALL Users

When any user logs in:

```
authenticate()  
  
JWT generated for THAT user
```

Every user gets unique token.

Example

User A Login

JWT contains:

```
user_id = 5
```

User B Login

JWT contains:

```
user_id = 22
```

Important

JWT is generated dynamically during login.

You do NOT manually create JWTs beforehand.

Real Production System

1 superadmin
few staff/admin users
thousands of normal users

All authenticate using same login system.

U You 5/24/2026, 4:32:12 AM

this line from `django.contrib.auth import authenticate` validates for the super user.

AI ChatGPT 5/24/2026, 4:32:13 AM

No.
It validates for ALL users.

```
from django.contrib.auth import authenticate
```

works for:

- superusers
- staff users
- normal users

What authenticate() Actually Does

It checks:

1. user exists?
2. password correct?
3. active account?

If correct:

returns that user object

Example

Suppose DB has:

admin	superuser
rahul	normal user
aman	normal user
staff1	staff user

All can use:

```
authenticate(username,password)
```

Difference Is Here

After authentication:

```
user.is_superuser
```

or

```
user.is_staff
```

Example

Superuser

```
is_superuser = True
```

Normal User

```
is_superuser = False
```

JWT Does Not Care

JWT only says:

```
"This user is valid"
```

Then YOU decide:

```
admin dashboard?  
student dashboard?  
employee dashboard?
```

based on role/permissions.

Simple Flow

authenticate()

returns User object

generate JWT

check permissions/roles

U You 5/24/2026, 4:33:23 AM

understand the work flow:

student id in db

student pass in db

db is postgres.

i will authenticate if the credentials in it are correct or not

then shall i use authenticate()

AI ChatGPT 5/24/2026, 4:33:24 AM

Yes.

Exactly.

Your workflow is correct.

Your System

PostgreSQL DB

Stores:

```
student_id  
password  
name  
email  
etc
```

Login Flow

```
Student enters:  
  student_id  
  password
```

Then:

```
authenticate()
```

checks Django user table.

VERY IMPORTANT

authenticate() works ONLY with Django Auth Users.

Meaning:

```
django.contrib.auth.models.User
```

or custom Django user model.

So You Have 2 Cases

CASE 1 — Recommended

Students are Django Users

Example:

```
username = student_id  
password = stored hashed password
```

Then YES:

```
authenticate()
```

is correct.

This is the BEST approach.

CASE 2 — Separate Student Table

Example:

```
Student table:  
  roll_no  
  password
```

NOT linked to Django auth system.

Then:

```
authenticate()
```

will NOT work automatically.

You must:

1. manually query DB
2. compare password
3. generate JWT manually

Recommended Architecture

Use Django User Model.

Example:

```
Student logs in using:  
username = student_id  
password
```

Then:

```
authenticate()
```

works perfectly.

Internal Flow

PostgreSQL

Django User table

authenticate()

User object

JWT generated

Best Practice in Real Projects

Usually:

Django User Table

Stores:

login credentials

permissions

roles

Student Table

Stores:

course
semester
marks
department
attendance

And linked like:

Student

User

using:

OneToOneField

Final Answer

YES:

If student credentials are stored using Django User model:

```
authenticate()
```

is the correct way.

U You 5/24/2026, 4:34:25 AM

just some handholding for this:

```
def student_login(request):  
    if request.method=="POST":  
        student_id = request.POST.get("student_id")  
        password = request.POST.get("password")
```

```
    try:  
        student = Student.objects.get(student_id=student_id)  
  
        #temp pwd check  
  
        if password == student.student_id:  
            #store into logs of activity  
            ActivityLogs.objects.create(  
                student=student,  
                action='Login'  
            )  
  
        return render(request,  
            'student_module/student_dashboard.html',{
```

```
        'student':student
    }
    )
else:
    return render(request,
        'student_module/login.html',{
            'error':'Invalid Password'
        }
    )

except Student.DoesNotExist:
    return render(request,
        'student_module/login.html',
        {
            "error":'Student id invalid'
        }
    )

return render(request, 'student_module/login.html')
```

- dont generate all at once , teach me what to do step by step

 **ChatGPT** 5/24/2026, 4:34:27 AM

Good.

Your current flow is already logically correct.

Now we will ONLY focus on:

where JWT should come

Do NOT rewrite everything.

Step 1 — Understand Current Flow

Right now:

student id entered

DB lookup

password check

dashboard render

Correct.

Step 2 — Problem In Current System

Currently after login:

```
return render(...)
```

directly opens dashboard.

Meaning:

NO authentication state stored

If user refreshes page:

system does not truly know user logged in

Step 3 — JWT Purpose

After successful password verification:

create identity token

That token proves:

```
this student already authenticated
```

Step 4 — WHERE JWT COMES

Exactly HERE:

```
if password == student.student_id:
```

AFTER this line.

Because this means:

```
student verified successfully
```

ONLY THEN generate JWT.

Step 5 — Understand Mentally First

After successful login:

```
student verified
```

generate JWT

store token

allow dashboard

Step 6 — What JWT Will Contain

Usually:

student_id
student primary key
expiry time

NOT full student data.

Step 7 — Then Future Requests Work Like

dashboard request

token checked

student identified

allow access

Step 8 — Your Current Thinking Mistake

Right now you are thinking:

login success = dashboard open

But actual architecture is:

login success

identity token created

future pages trust token

Step 9 — First Task For You

Before writing JWT code:

understand THIS deeply:

JWT is NOT login.
JWT is proof that login already happened.

Step 10 — Next Tiny Step

Now next thing YOU should do:

learn this single line only:

```
RefreshToken.for_user(user)
```

Question you should ask yourself:

```
What is "user" in my Student model case?
```

THAT is the next understanding point.

U You 5/24/2026, 4:35:41 AM

ok for all steps , make the step wise code for me for teaching, dont make all at once

AI ChatGPT 5/24/2026, 4:35:42 AM

Perfect.
We will do ONLY one small step now.

STEP 1 — Install Import

Open:

```
views.py
```

Add ONLY this import first:

```
from rest_framework_simplejwt.tokens import RefreshToken
```

Understand Why

This class:

```
RefreshToken
```

is the JWT generator.

It creates:

refresh token
access token

Your Current Flow

Currently:

student verified

direct dashboard render

What We Will Soon Change Into

student verified

JWT generated

token stored

dashboard render

VERY IMPORTANT

Do NOT write token code yet.

First only:

```
import successfully
```

Then tell me:

```
done
```

U **You** 5/24/2026, 4:36:52 AM

```
done
```

AI **ChatGPT** 5/24/2026, 4:36:53 AM

Good.

Now next tiny step.

STEP 2 — Generate JWT

Inside this block:

```
if password == student.student_id:
```

ADD this line first:

```
refresh = RefreshToken.for_user(student)
```

So temporarily it becomes:

```
if password == student.student_id:  
  
    refresh = RefreshToken.for_user(student)  
  
    #store into logs of activity
```

NOW STOP CODING

Understand this line.

What This Does

This line says:

```
Create JWT token for this authenticated user
```

IMPORTANT QUESTION

Will this work right now?

Most probably:

```
NO
```

WHY?

Because:

```
student is NOT Django Auth User model
```

SimpleJWT expects:

Django User object

usually:

```
from django.contrib.auth.models import User
```

BIG UNDERSTANDING

Your current Student model is probably:

custom table

NOT Django authentication system.

So What Will Happen?

Likely error:

```
'Student' object has no attribute 'id'
```

or authentication-related issue.

THIS IS GOOD

Because now you are discovering:

JWT is tightly connected to Django auth system

Your Next Thinking Task

You now must decide:

OPTION A — Recommended

Connect Student with Django User model.

Architecture:

Django User

linked to

Student Profile

BEST practice.

OPTION B

Keep separate Student table and manually build JWT.

Harder.

For Learning

I strongly recommend:

OPTION A

Your Next Step

Do NOT code more.

First tell me:

show Student model

Then I will guide next tiny step.

U You 5/24/2026, 4:40:08 AM

here is appStudent > [models.py](#)

```
from django.db import models
from appErpAdmin.models import CourseSessions
```

Create your models here.

```
class Student(models.Model):
    name = models.CharField(max_length=25)
    student_id = models.CharField(max_length=12,unique=True)
    course = models.CharField(max_length=6)
    email = models.EmailField(unique=True)
    session= models.CharField(max_length=10)
    image=models.ImageField(upload_to='students/')
    date_created = models.DateTimeField(auto_now_add=True)

class ActivityLogs(models.Model):
```

```
    ACTION_CHOICES = [
        ('Login', 'Login'),
        ('Logout', 'Logout'),
    ]

    student = models.ForeignKey(
        Student,
        on_delete=models.CASCADE
    )
```

```
action = models.CharField(
    max_length=10,
    choices=ACTION_CHOICES
)

timestamp = models.DateTimeField(
    auto_now_add=True
)

def __str__(self):
    return f"{self.student.student_id} - {self.action}"
```

tell me what to do now

 **ChatGPT** 5/24/2026, 4:40:09 AM

Good.
Now the architecture is clear.
Your Student model is:

NOT connected to Django User model

So:

RefreshToken.for_user(student)

will NOT work correctly.

What To Do NOW

We will do proper architecture.

STEP 3 — Add Django User Link

First import User model.

In models.py

ADD:

```
from django.contrib.auth.models import User
```

Understand Why

Django JWT system works with:

```
User objects
```

NOT custom student objects directly.

Final Architecture We Want

Django User

linked to

Student Profile

This is industry standard.

STEP 4 — Add OneToOneField

Inside Student model,

ADD this field near top:

```
user = models.OneToOneField(  
    User,  
    on_delete=models.CASCADE,  
    null=True  
)
```

Understand This Deeply

Now each student gets:

one Django login account

linked with:

one student profile

Architecture After This

User Table

id

username

password

linked

Student Table

name

student_id

course
email

Why This Is Powerful

Now:

```
authenticate()
```

can work perfectly.

AND:

```
JWT works perfectly
```

NEXT

After adding field:

run:

```
python manage.py makemigrations
```

then:

```
python manage.py migrate
```

STOP THERE

Do NOT change login logic yet.

First do:

1. import User
2. add OneToOneField
3. migrate

Then say:

```
done
```

U **You** 5/24/2026, 4:44:21 AM

done

Perfect.

Now your architecture became correct.

Next step:

connect actual login credentials

STEP 5 — Understand New Situation

Now every Student can have:

linked Django User account

Meaning:

Student profile data
+
Django authentication system

STEP 6 — What You Must Do NOW

For every student:

create a Django User.

Example mentally:

User:

username = student_id

password = secret123

linked to

Student:

name = Rahul

course = MCA

VERY IMPORTANT

Now:

```
authenticate()
```

will validate:

```
username + password
```

using Django auth system.

STEP 7 — Temporary Manual Testing

For ONE student first.

Open Django shell:

```
python manage.py shell
```

STEP 8 — Import Models

Inside shell:

```
from django.contrib.auth.models import User
```

```
from appStudent.models import Student
```

STEP 9 — Get Student

Example:

```
student = Student.objects.first()
```

STEP 10 — Create Django User

Example:

```
user = User.objects.create_user(  
    username=student.student_id,  
    password='1234'  
)
```

Understand VERY Carefully

NEVER do:

```
password='plain text in db'
```

directly.

`create_user()` automatically:

```
hashes password securely
```

STEP 11 — Link Student With User

```
student.user = user
```

```
student.save()
```

FINAL RESULT

Now:

```
Student object  
  linked  
Django User object  
  supports  
  authenticate()  
  JWT
```

permissions
sessions

STOP HERE

Do NOT change login view yet.

First create ONE linked user successfully.

Then tell me:

done

U You 5/24/2026, 4:49:43 AM

done

AI ChatGPT 5/24/2026, 4:49:45 AM

Excellent.

Now finally:

real authentication starts

STEP 12 — Import authenticate

Open views.py

Add:

```
from django.contrib.auth import authenticate
```

Understand

Previously YOU were checking:

```
if password == student.student_id
```

which is temporary and insecure.

Now Django will check:

```
hashed password properly
```

STEP 13 — Replace Password Check Logic

Find this block:

```
if password == student.student_id:
```

REMOVE it.

Replace With

```
user = authenticate(  
    username=student_id,  
    password=password  
)
```

Understand Carefully

This means:

```
check username/password against Django User table
```

NOT Student table directly.

What Happens Internally

Django does:

```
find user
```

```
compare hashed password
```

```
return User object if valid
```

STEP 14 — Next Condition

Now BELOW authenticate:

write:

```
if user is not None:
```

Understand Meaning

credentials valid

Else:

credentials invalid

Your Flow Is Becoming

student enters credentials

authenticate()

Django validates securely

returns User object

JWT can now be generated

STOP HERE

Do NOT add JWT yet.

First make sure:

```
authenticate() works
```

Then tell me:

```
done
```

U **You** 5/24/2026, 4:56:32 AM

```
done
```

AI **ChatGPT** 5/24/2026, 4:56:33 AM

Perfect.

Now we will connect:

```
successful authentication
```


STEP 15 — Generate JWT

Inside:

```
if user is not None:
```

ADD:

```
refresh = RefreshToken.for_user(user)
```

Understand

This line creates:

```
refresh token  
access token
```

for authenticated user.

STEP 16 — Extract Access Token

Below that:

```
access_token = str(refresh.access_token)
```

Understand

JWT object becomes string like:

```
eyJhbGciOi...
```

STEP 17 — Temporarily Print Token

Below that:

```
print(access_token)
```

WHY?

First SEE JWT working.

Do not store yet.

Do not overcomplicate yet.

What Will Happen

Login success:

```
authenticate()
```

```
JWT generated
```

```
printed in terminal
```

STEP 18 — Test

Run server.

Login.

See terminal.

You should see huge token like:

eyJ0eXAiOiJKV1QiLCJhbGc...

VERY IMPORTANT UNDERSTANDING

That long string is:

digital proof of identity

Server can later verify:

Was this token genuinely issued by me?

using cryptographic signature.

STOP HERE

Do NOT store token yet.

First confirm:

JWT generation works

Then say:

done

U You 5/24/2026, 5:01:08 AM

done

AI ChatGPT 5/24/2026, 5:01:10 AM

Excellent.

Now you have officially generated JWT.

Next:

store authentication state

STEP 19 — Store Token

Below:

```
print(access_token)
```

ADD:

```
request.session['access_token'] = access_token
```

Understand

Currently we are storing JWT inside:

```
Django session
```

ONLY for learning.

Later you will learn:

- cookies
 - Authorization headers
 - localStorage
-

STEP 20 — Why Store Token?

Without storing:

JWT generated
immediately lost

Then future pages cannot know:

user already authenticated

STEP 21 — New Flow

Now your system becomes:

login
authenticate()
JWT generated
stored in session

dashboard allowed

STEP 22 — Protect Dashboard

Now we must make dashboard trust JWT.

Open your dashboard view.

At TOP of dashboard view:

```
token = request.session.get('access_token')
```

Understand

This asks:

```
Does user already have JWT token?
```

STEP 23 — Add Protection

Below that:

```
if not token:  
    return redirect('student_login')
```

Meaning

If no token:

```
not authenticated
```

send back to login.

FINAL FLOW NOW

```
Student logs in
```

```
credentials verified
```

```
JWT generated
```

stored in session

dashboard checks token

access granted

IMPORTANT

Currently dashboard only checks:

token exists?

NOT:

token valid?

expired?

tampered?

That comes next.

U **You** 5/24/2026, 5:06:31 AM

done

 **ChatGPT** 5/24/2026, 5:06:34 AM

Perfect.

Now you officially have:

basic JWT authentication flow

working.

Now next important understanding:

CURRENTLY Your System Is Weak

Right now dashboard only checks:

if token exists

This means:

fake token text could also pass

because we are NOT verifying JWT yet.

STEP 24 — Understand JWT Verification

JWT has 3 parts:

```
HEADER.PAYLOAD.SIGNATURE
```

Server verifies:

```
Was this token generated by me?  
Is it expired?  
Was it modified?
```

STEP 25 — Import JWT Backend

In views.py

ADD:

```
from rest_framework_simplejwt.authentication import JWTAuthentication
```

Understand

This class verifies token authenticity.

STEP 26 — Dashboard Protection Upgrade

Inside dashboard view.

Currently:

```
token = request.session.get('access_token')
```

Keep it.

BELOW IT ADD

```
jwt_auth = JWTAuthentication()
```

Understand

This creates JWT verifier object.

STEP 27 — Verify Token

Below that:

```
validated_token = jwt_auth.get_validated_token(token)
```

Understand

This checks:

```
signature valid?  
expired?  
tampered?
```

If Token Invalid

System throws exception.

Meaning:

```
dashboard access denied
```

BIG UNDERSTANDING

Now your architecture becomes REAL JWT auth:

login

JWT issued

future request carries JWT

JWT verified cryptographically

access granted

STOP HERE

Do NOT add try-except yet.

First test.

Login again.

Then open dashboard.

See if errors happen.

Tell me exactly what happens.

U You 5/24/2026, 5:11:45 AM

access was granted

AI ChatGPT 5/24/2026, 5:11:47 AM

Excellent.

Now your JWT is actually being validated.

You have crossed the biggest conceptual hurdle.

What Your System NOW Does

Student logs in

Django authenticate()

JWT generated

JWT stored

Dashboard receives JWT

JWT signature verified

Access granted

VERY IMPORTANT

You are now doing REAL authentication.

Not fake login.

Current Weakness

If token becomes invalid:

```
expired  
modified  
deleted
```

dashboard will crash with exception.

NEXT STEP

Now we add:

```
safe handling
```

using:

```
try:  
except:
```

What This Will Do

Instead of server error:

```
redirect user back to login
```

when token invalid.

YOUR NEXT THINKING TASK

Before coding:

understand this deeply:

```
JWT verification itself IS the authentication check.
```

Not session.

Not dashboard access.

The REAL check is:

```
get_validated_token()
```

because that proves:

```
token genuine  
issued by server  
not modified  
not expired
```

You Have Now Learned

- Django auth
- custom student system
- User linking
- JWT generation
- JWT storage
- JWT verification
- protected routes
- authentication workflow

This is already beyond beginner level understanding.

U **You** 5/24/2026, 5:12:32 AM

now id stu001 has password 1234,
what about id stu002, that has not been linked to user,?

 **ChatGPT** 5/24/2026, 5:12:34 AM

Then stu002 cannot login using:

```
authenticate()
```

because Django auth system knows NOTHING about that student yet.

Current Situation

stu001

Has:

```
Student object  
+  
linked Django User
```

So:

```
authenticate()
```

works.

JWT works.

stu002

Only has:

```
Student object
```

NO linked Django User.

So:

```
authenticate()
```

fails.

returns:

```
None
```

Why?

Because authenticate checks:

Django User table

NOT Student table.

Current Architecture

Student Table

stu001	linked to User
stu002	not linked
stu003	not linked

Therefore

Only linked students can:

login
generate JWT

access protected routes

Real Production Approach

When student created:

system automatically:

1. creates Student
2. creates Django User
3. links both

Immediately.

IMPORTANT UNDERSTANDING

You now have TWO systems:

Student Profile System

course
name

email
attendance

Authentication System

username
password
JWT
permissions

And both are connected.

Your Next Realization

The actual login credentials belong to:

Django User model

NOT Student model anymore.

Student model becomes:

profile/business data

U You 5/24/2026, 5:46:22 AM

i want to link all database records.

```
from django.contrib.auth.models import User
from appStudent.models import Student
```

```
student = Student.objects.first()
```

but i want a for each to bring all the objects.

Now one by one i pick and assign them to user table for now keeping their password same as id.

pick an object from all objects.

check if present in the user table (authentication)

if yes then skip to next object of student

and check if present in user table.

if not then

```
user = User.objects.create_user( username=student.student_id, password=student.student_id)
```

```
student.user = user
student.save()
```

AI ChatGPT 5/24/2026, 5:46:23 AM

Yes.

Your thinking is correct.

Workflow:

```
loop all students  
  
check if Django User already exists  
  
if exists:  
    skip  
else:  
    create User  
    link Student
```

STEP 1 — Get All Students

Instead of:

```
Student.objects.first()
```

use:

```
students = Student.objects.all()
```

Understand

This gives:

```
all student records
```

STEP 2 — Loop Through Students

Now:

```
for student in students:
```

Understand

This picks:

```
one student at a time
```

Example mentally:

```
stu001  
stu002  
stu003  
...
```

STEP 3 — Check Existing User

Inside loop:

```
User.objects.filter(  
    username=student.student_id  
)exists()
```

Understand

This asks:

```
Does auth user already exist?
```

STEP 4 — If Exists

```
if user_exists:  
    continue
```

Meaning

```
skip current student  
move to next
```

STEP 5 — If NOT Exists

Then:

```
create_user()
```

STEP 6 — Link Student

Then:

```
student.user = user  
student.save()
```

FINAL FLOW

```
pick student
```

```
check auth table
```

```
already exists?
```

```
  yes  skip
```

```
  no
```

```
create auth user
```

```
link student
```

```
save
```

VERY IMPORTANT

This is basically:

```
database migration into authentication system
```

which is extremely common in real projects.

AFTER THIS

ALL students will support:

```
authenticate()  
JWT  
permissions  
protected routes
```

U **You** 5/24/2026, 5:52:36 AM

```
for student in students:  
    ... User.objects.filter(username=student.student_id).exists()  
    ... if user_exists:  
        ... continue  
    ... create_user()
```

```
... student.user = user
... student.save()
...
False
Traceback (most recent call last):
File "", line 3, in
NameError: name 'user_exists' is not defined
```

 **ChatGPT** 5/24/2026, 5:52:39 AM

Good.

You understood the logic, but missed variable assignment.

This line:

```
User.objects.filter(username=student.student_id).exists()
```

returns:

```
True or False
```

But you did NOT store it into:

```
user_exists
```

Correct Tiny Step

Inside loop:

```
user_exists = User.objects.filter(  
    username=student.student_id  
)exists()
```

Understand

Now:

```
user_exists
```

contains:

```
True  
or  
False
```

THEN

You can do:

```
if user_exists:  
    continue
```

Your Mistake

You wrote:

```
if user_exists:
```

before creating variable.

So Python said:

```
NameError
```

because variable does not exist yet.

NEXT

Now do ONLY:

```
user_exists = ...
```

inside loop.

Then print it first.

Example mentally:

```
stu001  True
stu002  False
```

First verify that understanding.

U You 5/24/2026, 5:57:33 AM

ok make the code for above

AI ChatGPT 5/24/2026, 5:57:35 AM

```
for student in students:
```

```
user_exists = User.objects.filter(  
    username=student.student_id  
)exists()
```

```
if user_exists:  
    continue
```

```
user = User.objects.create_user(  
    username=student.student_id,  
    password=student.student_id  
)
```

```
student.user = user
```

```
student.save()
```